

C语言程序设计

第八讲 指针



张 华

主要内容

- 指针的概念
- 指针变量的定义和初始化
- 指针的指针
- 指针运算符
- 指针作为函数参数
- 指针与数组
- 指针数组
- 函数指针
- 程序设计举例

简介

指针

- ✿ 功能强大，不易掌握。
- ✿ 用来模拟引用传递。
- ✿ 与数组和字符串关系密切。
- ✿ 可以创建和操作动态的数据结构：
 - 链表
 - 队列
 - 栈
 - 树

指针的概念

指针就是内存对象的地址。

✿ 内存对象包括：变量，数组，函数等。

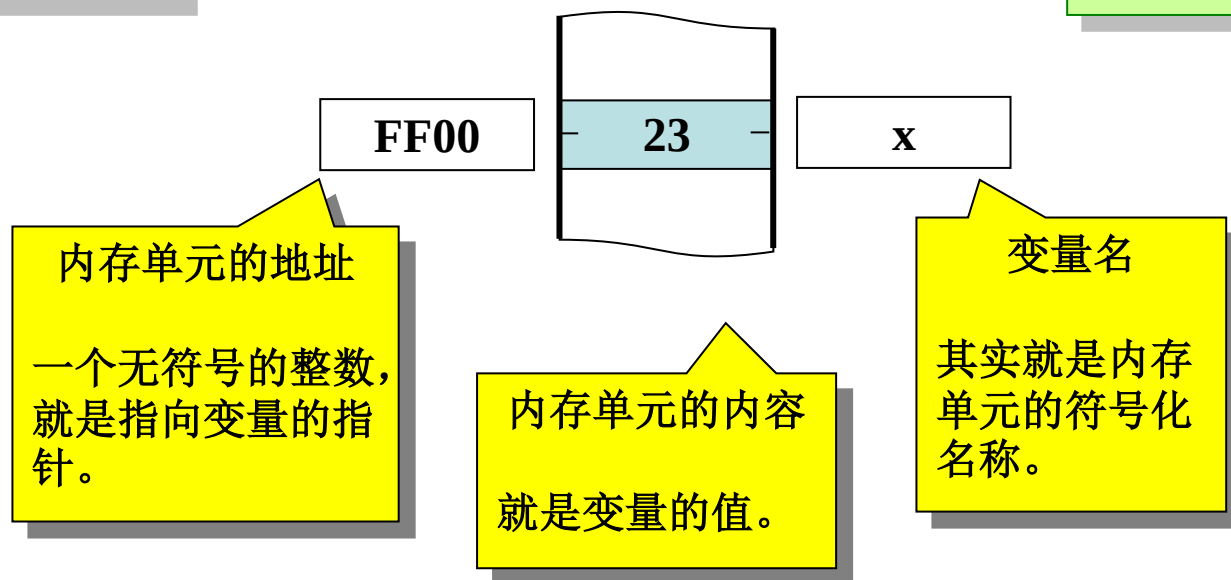
✿ C语言允许直接通过地址来处理数据。

```
int x;  
x=23;
```

直接引用

Direct reference

通过变量名直接引用
变量占用的内存单元。



内存对象的地址

变量的地址

- ✿ 用取地址运算符（&）获得变量在内存中的地址。

```
int var;  
scanf("%d", &var);
```

数组的地址

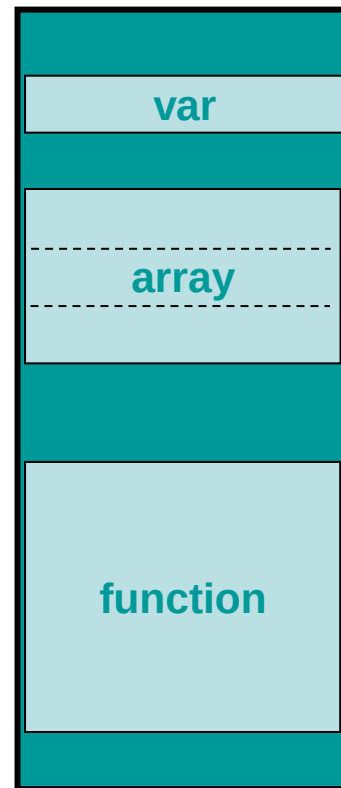
- ✿ 即第一个元素的地址，用数组名表示。

```
int array[3];
```

函数的地址

- ✿ 用函数名表示。

```
int function(int x);
```



指针变量

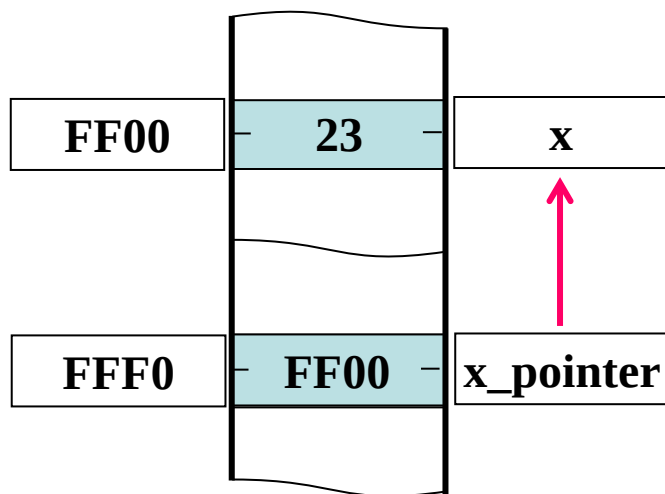
指针变量就是保存指针（内存地址）的变量。

```
int x=23;  
int *x_pointer;  
x_pointer=&x;
```

间接引用

Indirect reference

通过指针变量间接引用
变量占用的内存单元。



指针变量

- 保存变量的地址。
- 变量 `x_pointer` 的值是变量 `x` 的地址（指针）。
- 目前，指针 `x_pointer` 指向变量 `x`。

指针变量

指针变量的声明

类型说明符 * 指针变量名;

```
int *x_pointer;
```

- * 表示 **x_pointer** 是一个指针变量
- **x_pointer** 的值是 **int*** 类型的指针，读
 - 指向 **int** 型数据的指针
 - 指向整型对象的指针
- 指针变量可以声明为指向任何类型的数据（或对象）

✿ 声明多个指针变量时，每个变量名前都必须有 *

```
int *x_pointer, *y_pointer;  
char *charPtr;
```

注意

所有指针变量保存的数据的类型是相同的，即一个内存单元的地址，但是，它们指向的数据的类型可以不同。

指针变量

指针变量的初始化

✿ 在声明语句中为指针变量指定初值。

- 指针变量可以被初始化为 **0**，**NULL** 或 一个地址量。
 - **0** 和 **NULL** 是等价的（用**NULL**更好）
 - **NULL** 是在<stdio.h>等几个头文件中定义的符号常量，表示0

```
int x, *p=NULL;
```

空指针：不指向任何对象

```
int x, *p=&x;
```

```
int x, *p=0xffe;
```

Warning: Nonportable pointer
conversion

```
int x, *p=(int *)0xffe;
```

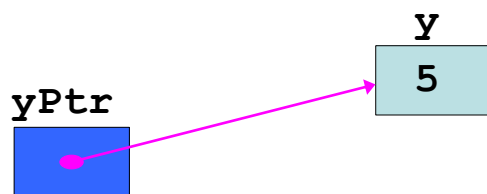
强制类型转换

取地址运算符

取地址运算符：&

✿ 返回变量在内存中的地址。

```
int y, *yPtr;  
y = 5;  
yPtr = &y;
```



yPtr “指向” y



yPtr的值是y的地址

指针运算符

指针运算符：*

- 返回指针变量所指向的对象的别名（间接引用）。

```
int y=5, *yPtr;  
yPtr = &y;
```

➤ ***yPtr** 就是 **y**，因为 **yPtr** 指向 **y**。

- 可以用在赋值语句中。

```
int y, *yPtr;  
yPtr = &y;  
*yPtr = 5;
```

案例分析：指针运算符

指针运算符

```
#include <stdio.h>
void main() {
    int a, *aPtr;
    a = 7;
    aPtr = &a;

    printf("The address of a is %p"
           "\nThe value of aPtr is %p", &a, aPtr);
    printf("\n\nThe value of a is %d"
           "\nThe value of *aPtr is %d", a, *aPtr);
    printf("\n\nShowing that * and & are inverses of each other."
           "\n&*aPtr = %p"
           "\n*&aPtr = %p", &*aPtr, *&aPtr);
}
```

案例分析：指针运算符

指针运算符

运行结果

```
The address of a is 1A58  
The value of aPtr is 1A58
```

```
The value of a is 7  
The value of *aPtr is 7
```

```
Showing that * and & are inverses of each other.  
&*aPtr = 1A58  
*&aPtr = 1A58
```

* 和 & 是互反的

指针的指针

指向指针的指针

- 由于指针变量也是一个变量，它也有存储地址，因此，同样可以定义另一个指针变量指向该指针，我们称该指针为“指针的指针”，也可称为“二维指针”。

```
#include <stdio.h>

int main() {
    int x=123, *p=&x, **pp=&p;

    printf("%p\n", p);
    printf("%p\n", pp);

    printf("%d\n", *p);
    printf("%d\n", **pp);

    return 0;
}
```

指针作为函数的参数

指针作为函数的参数用来模拟引用传递。

- 用指针变量作为函数的形式参数。

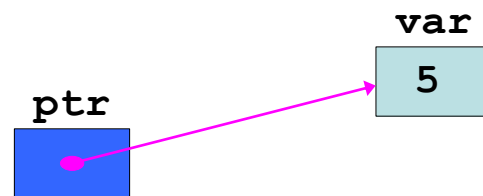
```
int myFunc( int *ptr );
```

- 调用函数时，把变量的地址作为实际参数传递给函数。

```
myFunc( &var );
```

- 在被调用函数中，用 * 运算符间接引用变量。

```
int myFunc( int *ptr )  
{  
    *ptr = ...  
}
```



案例分析：指针参数

问题：计算任意整数的立方。

✿ 值传递：callByValue()

```
#include <stdio.h>
int callByValue(int); /*返回参数的立方*/

void main() {
    int number = 5;
    printf("The original value of number is %d", number);
    number = callByValue(number);
    printf("\nThe new value of number is %d", number);
}

int callByValue(int n) {
    return n*n*n;
}
```

```
The original value of number is 5
The new value of number is 125
```

案例分析：指针参数

问题：计算任意整数的立方。

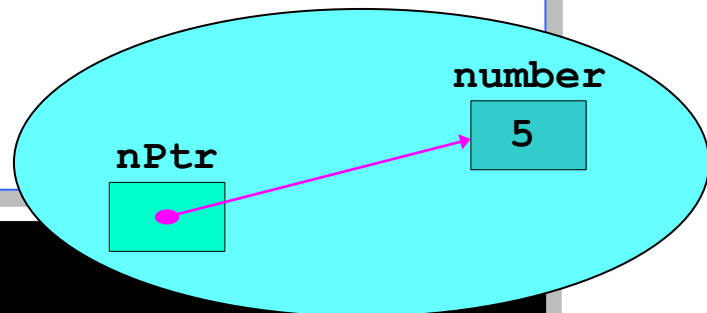
✱ 值传递：callByReference()

```
#include <stdio.h>
void callByReference(int*);

void main() {
    int number = 5;
    printf("The original value of number is %d", number);
    callByReference(&number);
    printf("\nThe new value of number is %d", number);
}

void callByReference(int *nPtr) {
    *nPtr = *nPtr**nPtr**nPtr;
}
```

```
The original value of number is 5
The new value of number is 125
```



案例分析：指针参数

 **问题：计算任意整数的立方。**

 **结果分析**

调用前

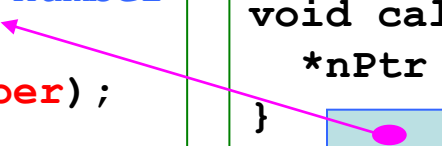
```
void main() {
    5 number
    int number = 5;
    callByReference(&number);
}
```

```
void callByReference(int *nPtr) {
    *nPtr = *nPtr**nPtr**nPtr;
}    nPtr
```

调用时

```
void main() {
    125 number
    int number = 5;
    callByReference(&number);
}
```

```
void callByReference(int *nPtr) {
    *nPtr = *nPtr**nPtr**nPtr;
} ● nPtr
```



调用后

```
void main() {
    125 number
    int number = 5;
    callByReference(&number);
}
```

```
void callByReference(int *nPtr) {
    *nPtr = *nPtr**nPtr**nPtr;
}    nPtr
```

案例分析：指针参数

问题：交换两个变量的值。

哪一种实现是正确的？

```
void swap(int x, int y) {  
    int tmp;  
    tmp=x; x=y; y=tmp;  
}
```

```
void main() {  
    int a=4, b=6;  
    swap(a, b);  
    printf("a=%d,b=%d",a,b);  
}
```

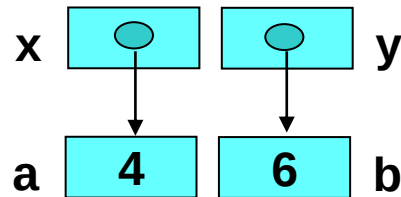
x 4 6 y

a 4 6 b

a=4,b=6

```
void swap(int *x, int *y) {  
    int tmp;  
    tmp=*x; *x=*y; *y=tmp;  
}
```

```
void main() {  
    int a=4, b=6;  
    swap(&a, &b);  
    printf("a=%d,b=%d",a,b);  
}
```



a=6,b=4

指针运算

指针可以参与以下运算：

✿ 赋值运算

- 给指针变量赋值

✿ 关系运算

- 两个指针之间的比较

✿ 算术运算

- 加（减）一个整数
- 两个指针相减

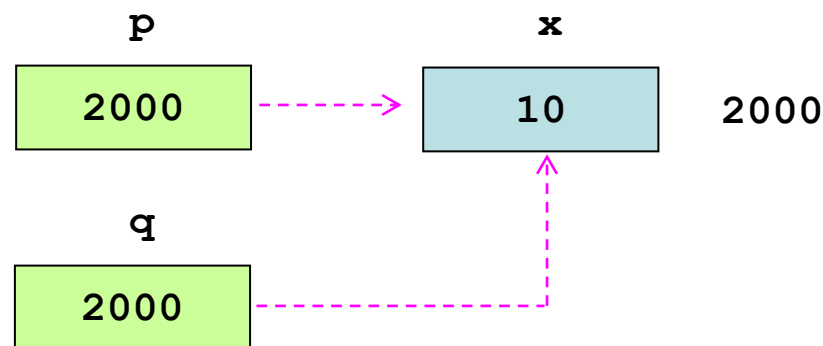
指针运算

指针的赋值运算

✿ 可以把指针赋给同类型的指针变量。

```
int x=10, *p, *q;  
p = &x;  
q = p;  
printf("*q=%d", *q);
```

```
*q=10
```



指针运算

指针的赋值运算

✿ 把指针赋值给类型不同的指针变量时要进行类型转换。

```
int a, *intPtr=&a;  
char *charPtr;
```

```
charPtr = (char*)intPtr;
```

✿ 但 **void *** 类型的指针是一个例外。

```
void *voidPtr;  
voidPtr = intPtr;  
charPtr = voidPtr;
```

void*

- 通用指针
- 代表任何指针类型
- 不能间接引用

常见的应用形式

```
void f(void *p);  
...  
f(ptr);
```

```
*voidPtr = 10;
```

指针运算

指针的关系运算

✿ 比较两个指针的值。

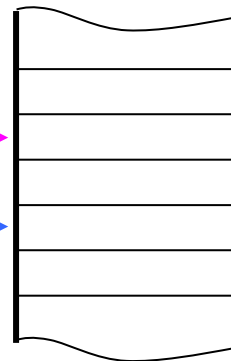
```
p1 < p2  
p1 <= p2  
p1 > p2  
p1 >= p2  
p1 == p2  
p1 != p2
```

p1

2000

p2

2002



```
char *charPtr, *ptr;  
...  
charPtr >= (char*)2000  
...  
if (ptr != 0)  
...
```

与同类型的指针常
量比较
常常与 0 比较

指针运算

指针的算术运算

- ✿ 自增自减 ($++$, $--$)
- ✿ 加上一个整数 ($+$, $+=$, $-$, $-=$)
- ✿ 两个指针相减

指针运算

指针的算术运算

举例

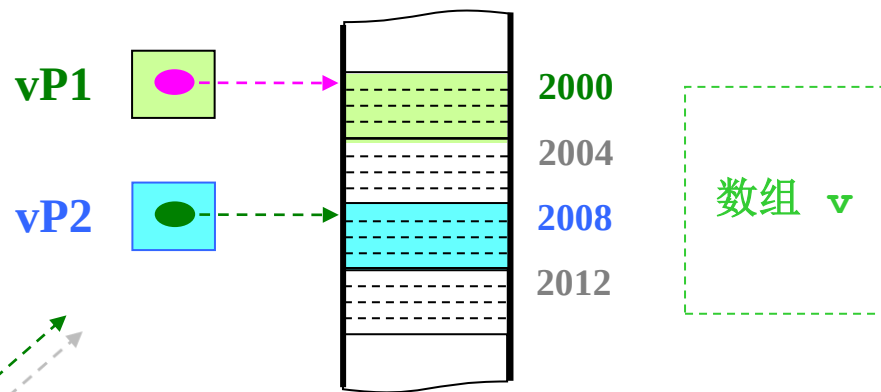
```
int *vP1, *vP2;  
vP1 = (int*)2000;  
vP2 = vP1 + 2;
```

? $vP1 + 2$

$2000 + 2 = 2002$

$2000 + 2 * 4 = 2008$

所指向的内存对象的
长度 $\text{sizeof}(\text{int})$

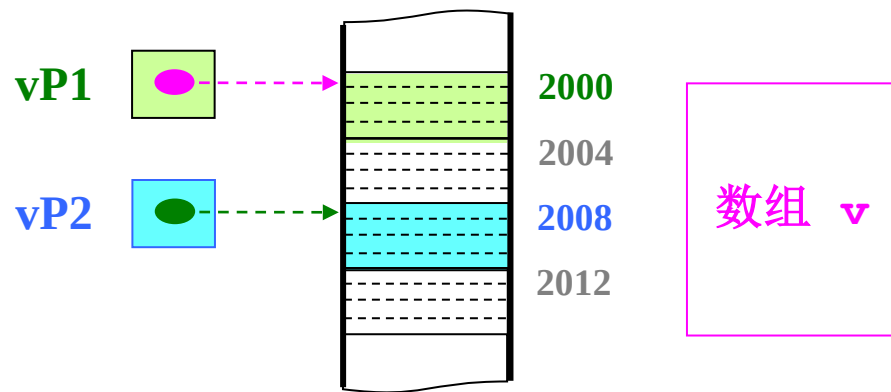


指针运算

指针的算术运算

举例

```
int *vP1, *vP2;  
vP1 = (int*)2000;  
vP2 = (int*)2008;
```



? `vP2 - vP1`

`=(2008-2000)/sizeof(int)`

`=2`

`vP2`和`vP1`之间内存对象的个数

指针的算术运算在数组上使用才有意义

指针与数组

数组和指针关系密切。

- ✱ 数组名是一个指针常量。
- ✱ 数组元素指针：指向数组元素的指针。
- ✱ 数组元素指针可以用来完成任何涉及数组下标的操作。

```
int b[5];  
int *bPtr;
```

➤ 将 **bPtr** 的值置为数组 **b** 中的第一个元素的地址

bPtr = b;

➤ 等价于

bPtr = &b[0];

指针与数组

■ 数组和指针关系密切。

✿ 引用数组元素的表达式

- 数组元素 **b[3]** 可以用 *** (bPtr + 3)** 来引用
 - 3是偏移量
 - 这种表示法称为指针偏移量表示法
- 还可以用 **bPtr[3]** 来引用
 - 称为指针下标表示法
 - 与 **b[3]** 相同
- 还可以用 ***(b + 3)** 来引用

指针与数组

数组和指针关系密切。

引用数组元素的表达式

下标法

```
main() {  
    int i, a[5]={1,2,3,4,5};  
    for (i=0; i<5; i++)  
        printf("%2d", a[i]);  
}
```

地址法

```
main() {  
    int i, a[5]={1,2,3,4,5};  
    for (i=0; i<5; i++)  
        printf("%2d", *(a+i));  
}
```

指针法

```
main() {  
    int a[5]={1,2,3,4,5}, *p;  
    for (p=a; p<(a+5); p++)  
        printf("%2d", *p);  
}
```

指针与数组

数组和指针互换使用时的注意事项

✿ 数组名是一个指针常量。

```
main() {  
    int a[5]={1,2,3,4,5}, *p;  
    for ( p=a; a<(p+5); a++ )  
        printf("%2d",*a);  
}
```

错

因为a是数组名，即数组的首地址，它的值不允许被修改！
是一个常量。

指针与数组

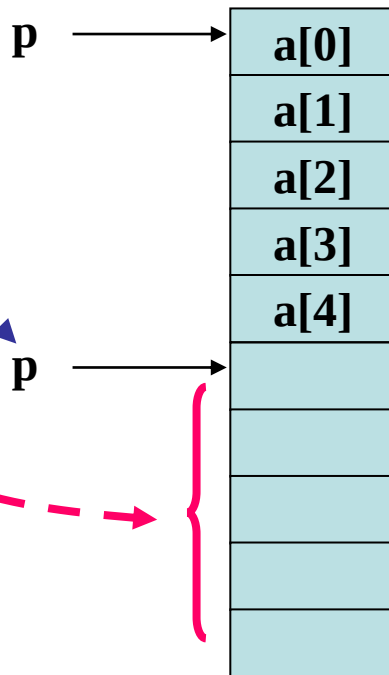
数组和指针互换使用时的注意事项

✿ 注意指针变量的值。

```
main() {  
    int i, a[5], *p;  
    p=a;  
    for ( i=0; i<5; i++ )  
        scanf( "%d", p++ );  
    for ( i=0; i<5; i++, p++ )  
        printf( "%d ", *p );  
}
```

错

p = a;



数组a

要注意指针变量的当前值。

指针与数组

数组和指针互换使用时的注意事项

✿ 注意运算符的优先级和结合性。

```
int a=0, *aPtr=&a;

printf("a=%d,aPtr=%p\n", a, aPtr);
printf("%d\n", *aPtr++);
printf("a=%d,aPtr=%p\n", a, aPtr);
```

```
a=0,aPtr=1BC4
0
a=0,aPtr=1BC8
```

* aPtr ++

等价

* (aPtr ++)

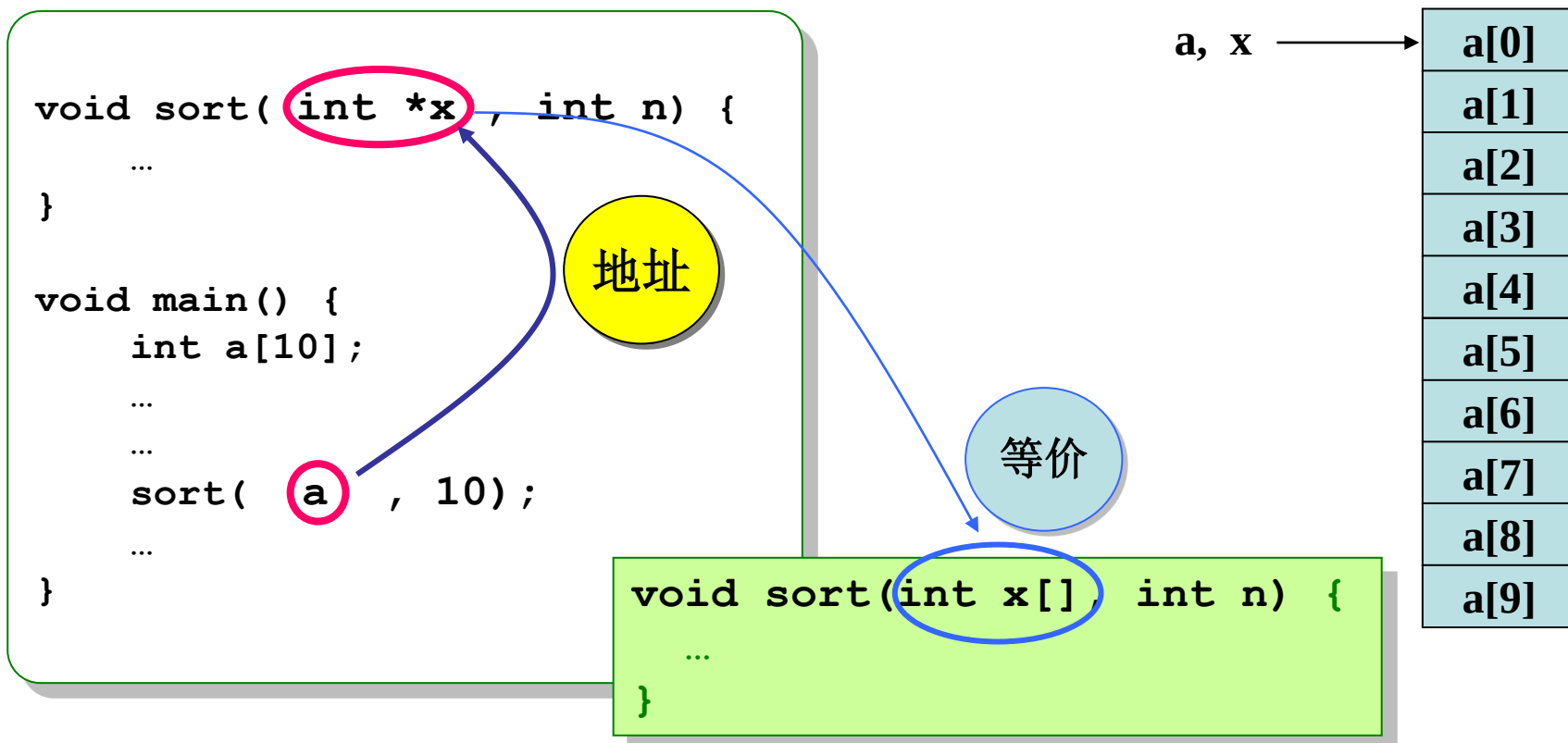
考虑:

*++aPtr 与 *(++aPtr)

指针与数组

数组参数的值传递

✿ 形参数组是一个指针变量。



指针与数组

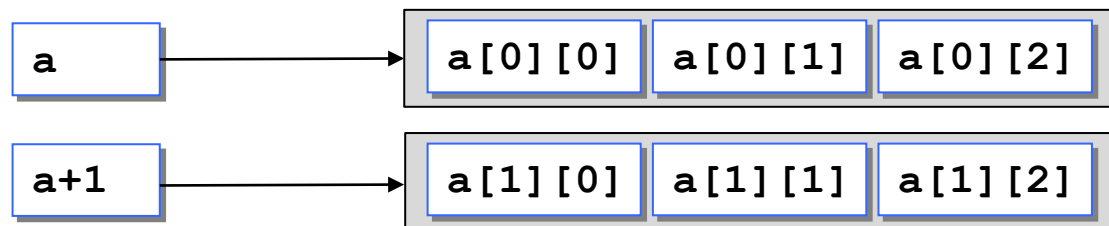
二维数组的指针

二维数组有两种地址（指针）

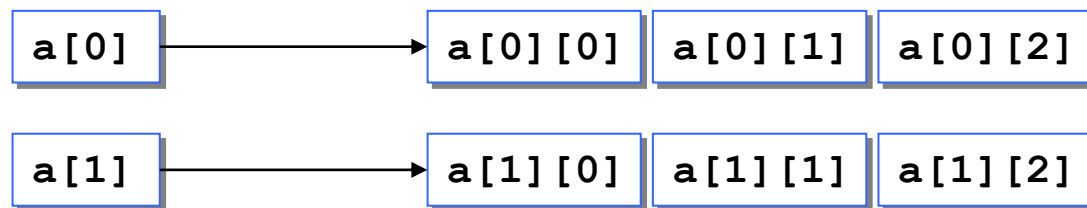
- 行地址（一维数组的地址）：每一行的地址
- 列地址（元素地址）：每一个元素的地址

```
int a[2][3];
```

行地址



列地址



指针与数组

二维数组的指针

二维数组有两种地址（指针）

```
int a[2][3]={1,2,3,4,5,6};

{
    int (*p_row)[3];
    p_row = a;           // a是行指针，指向第一行
}

{
    int *p_item;
    p_item = a[0];       // a[0]是元素指针，也是第一行的数组名
}

printf("%d\n", **a);     // *a 是第一行(数组)，元素指针
printf("%d\n", *a[0]);   // a[0]是第一行(数组)，元素指针
printf("%d\n", *(a[0]+1)); // a[0]+1是元素指针
printf("%p\n", a+1);     // a+1是行指针，指向第二行
printf("%p\n", a[0]+1);
```

```
1
1
2
0058FC30
0058FC28
```

指针与数组

二维数组做函数的参数

✿ 二维数组名做函数的实参，对应的形参是二维数组

```
void exchange(int (*a)[3], int (*b)[2], int m, int n)
//void exchange(int a[][3], int b[][2], int m, int n)
{
    ...
}

void main() {
    int a[2][3], b[3][2];
    ...
    exchange(a, b, 2, 3);
    ...
}
```

第一维的长度可以为空，
其他维的长度必须指定。

实参数组与形参数组必须同构。

案例：用指针做函数参数

问题

- 有一个班，4个学生，学3门课，计算总平均分数以及第 n 个学生的成绩。

```
double score[4][3]=  
    {  
        {65,67,70},  
        {60,100,107},  
        {90,101,90},  
        {99,100,91}  
    };
```

要求

- 定义函数average，求总平均成绩。
- 定义函数total_score，计算每位学生的总分。

案例：用指针做函数参数

实现

```
#include <stdio.h>

/*
    计算全部成绩的平均分
    参数p指向一个成绩，即成绩数组的第一个元素
    参数n是成绩个数
*/
double average(double *p, int n);

/*
    计算每个学生的总分
    参数p指向长度为3的一维数组，即成绩表的第一个学生成绩数组
    参数n是数组数量，即学生数
    参数total指向一个成绩，即一个保存总分的一维数组的第一个元素
*/
void total_score(double (*p)[3], int n, double *total);
```

案例：用指针做函数参数

实现

```
int main() {
    double score[4][3]= {{65,67,70},
                          {60,100,107},
                          {90,101,90},
                          {99,100,91}};

    double avg, double total[4];
    int i;

    avg = average(*score, 12); /*求12个分数的平均分*/
    printf("average = %.2f\n", avg);

    total_score(score, 4, total); /*计算每个学生的总分*/
    printf("total scores: ");
    for (i=0; i<4; i++) {
        printf("%-6.1f", total[i]);
    }

    return 0;
}
```

案例：用指针做函数参数

实现

```
double average(double *p, int n) {  
    double *p_end;  
    double sum=0;  
  
    p_end = p + n - 1;  
    for(; p<=p_end; p++)  
        sum = sum + (*p);  
  
    return sum / n;  
}
```

案例：用指针做函数参数

实现

```
void total_score(double (*p)[3], int n, double *total) {
    int i;

    for (i=0; i<n; i++, p++) {
        int sum=0;
        double *s = *p;
        int j;
        for (j=0; j<3; j++, s++) {
            sum += *s;
        }
        *total = sum;
        total++;
    }
}
```

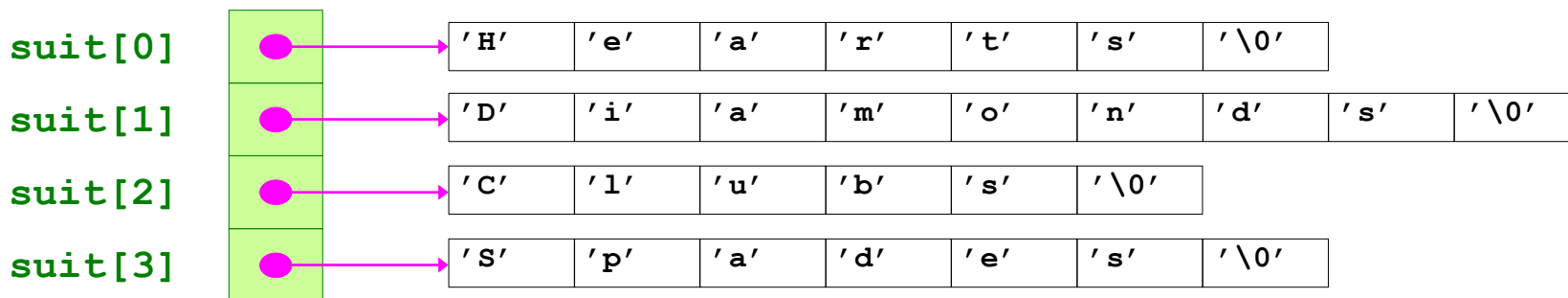

指针数组

数组元素是指针的数组

类型说明符 *数组名 [常量表达式];

✿ 常用来构造字符串数组。

```
char *suit[4] = {"Hearts", "Diamonds", "Clubs", "Spades"};
```



➤ 注意:

- 字符串并不在 **suit** 数组中。
- **suit** 数组只包含指向字符串的指针。

suit 的长度是固定的，但却可以访问任意长度的字符串

案例分析：指针数组

问题：洗牌和发牌的模拟

定义数据结构

- 字符串（指针）数组 **suit** 保存牌的花色名
- 字符串（指针）数组 **face** 保存牌的号码

		Ace	Two	Three	Four	Five	Six	Seven	Eight	Nine	Ten	Jack	Queen	King
		0	1	2	3	4	5	6	7	8	9	10	11	12
Hearts	0													
Diamonds	1													
Clubs	2													
Spades	3													

`deck[2][12]` represents the King of Clubs

Clubs

King

- 二维数组 **deck** 表示一副牌，行对应花色，列对应号码
 - 保存洗牌后牌的序号

案例分析：指针数组

问题：洗牌和发牌的模拟

设计算法

➤ 算法的顶部

对52张牌进行洗牌和发牌

➤ 第一次细化

初始化 `suit` 数组
初始化 `face` 数组
初始化 `deck` 数组
对 52 张牌洗牌
对 52 张牌发牌

案例分析：指针数组

问题：洗牌和发牌的模拟

✿ 设计算法

➤ 第二次细化

```
初始化 suit 数组  
初始化 face 数组  
初始化 deck 数组  
对 52 张牌中的每张  
    在随机选择的纸牌空位上放置纸牌序号  
对 52 张牌中的每张  
    查找 deck 数组中纸牌的序号，并显示纸牌的花色和号码
```

案例分析：指针数组

问题：洗牌和发牌的模拟

设计算法

第三次细化

初始化 `suit` 数组

初始化 `face` 数组

初始化 `deck` 数组

对 52 张牌中的每张

 随机选择纸牌位置

 当所选纸牌位置已经被选过（有序号）

 随机选择纸牌位置

 在所选纸牌位置中放置纸牌序号

对 52 张牌中的每张

 对于 `deck` 数组的每个位置

 如果该位置包含期望的纸牌序号

 显示纸牌的花色和号码

案例分析：指针数组

问题：洗牌和发牌的模拟

源代码

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void shuffle(int[][13]);
void deal(int[][13], char*[], char*[]);

void main() {
    char *suit[]={"Hearts", "Diamonds", "Clubs", "Spades"};
    char *face[]={"Ace","Two","Three",
                  "Four","Five","Six",
                  "Seven","Eight","Nine",
                  "Ten","Jack","Queen","King"};
    int deck[4][13]={0};
```

案例分析：指针数组

问题：洗牌和发牌的模拟

源代码

```
srand(time(0));  
  
shuffle(deck);  
deal(deck, face, suit);  
}
```

案例分析：指针数组

问题：洗牌和发牌的模拟

源代码

```
void shuffle(int wDeck[][13]) {  
    int row, column, card;  
  
    for (card=1; card<=52; card++) {  
        do {  
            row = rand()%4;  
            column = rand()%13;  
        } while (wDeck[row][column] != 0);  
        wDeck[row][column]=card;  
    }  
}
```


案例分析：指针数组

问题：洗牌和发牌的模拟

源代码

```
void deal(int wDeck[][13], char *wFace[], char *wSuit[]) {  
    int card, row, column;  
  
    for (card=1; card<=52; card++)  
        for (row=0; row<4; row++)  
            for (column=0; column<13; column++)  
                if (wDeck[row][column]==card)  
                    printf("%5s of %-8s%c",  
                        wFace[column], wSuit[row],  
                        card%2==0?'\\n':'\\t');  
}
```

函数指针

什么是函数指针

- ✿ 如果在程序中定义了一个函数，在编译时，编译系统为函数代码分配一段存储空间，这段存储空间的起始地址（又称入口地址）称为这个函数的指针。
- ✿ 可以定义一个指向函数的指针变量，用来存放某一函数的起始地址，这就意味着此指针变量指向该函数。

函数指针

函数指针变量调用函数

- 如果想调用一个函数,除了可以通过函数名调用以外,还可以通过指向函数的指针变量来调用该函数。

函数类型 (*标识符)(参数表)

```
#include <stdio.h>

int f(int x) { return x*x; }

int main() {
    int (*p)(int); /*定义函数指针变量*/
    p = f;
    p(123);

    return 0;
}
```

函数指针

函数指针作函数参数

- 函数指针变量常用的用途之一是把指针作为参数传递到其他函数。
- 指向函数的指针也可以作为参数，以实现函数地址的传递，这样就能够和被调用的函数中使用实参函数。

案例：函数指针的应用

函数指针的应用

✿ 输入两个整数和运算符 (+, -, *, /), 计算结果。

```
#include <stdio.h>

int add(int x, int y) { return x + y; }
int sub(int x, int y) { return x - y; }
int mul(int a, int b) { return a * b; }
int div(int a, int b) { return a / b; }

int f(int (*p)(int,int), int a, int b)
{
    return p(a,b);
}
```

案例：函数指针的应用

函数指针的应用

 续

```
int main()
{
    int a, b, result;
    char op;

    printf("Please input a and b:");
    scanf("%d%d", &a, &b);

    fflush(stdin); /*清空键盘缓冲区*/
```

案例：函数指针的应用

函数指针的应用

✿ 续

```
switch(op) {  
    case '+': result = f(add, a, b); break;  
    case '-': result = f(sub, a, b); break;  
    case '*': result = f(mul, a, b); break;  
    case '/': result = f(div, a, b); break;  
}  
  
printf("Result = %d\n", result);  
  
return 0;  
}
```

返回指针的函数

返回指针的函数

- ✿ 一个函数可以返回一个整型值、字符值、实型值等，也可以返回指针型的数据，即地址。
- ✿ 其概念与以前类似，只是返回值的类型是指针类型。
- ✿ 这种带回指针值的函数，一般定义形式为：
类型名 *函数名(参数表列);

案例：返回指针的函数

问题

- 有若干个学生的成绩，要求在用户输入学生序号以后，能输出该学生的全部成绩。用指针函数来实现。

```
#include <stdio.h>

float *search(float (*pointer)[4], int n);

int main() {
    float score[3][4]={ {60,70,100,90},
                        {56,109,67,1010}, {34,710,90,66}};

    float *p;
    int i, m;

    printf("enter the number of student: ");
    scanf("%d", &m);
```

案例：返回指针的函数

问题

续

```
printf("The scores of No. %d are: \n", m);  
p = search(score, m);  
for(i=0; i<4; i++)  
    printf("%5.2f\t", *(p+i));  
}  
  
float *search(float (*p)[4], int n) {  
    float *pt;  
    pt = *(p+n);  
    return pt;  
}
```

小结

- 指针变量的值都是一个内存地址，即该指针变量所指向的内存对象的地址。
- 指针加上一个整数或进行自增运算时，指针值的改变都是以所指向对象的字节数为单位的。
- 数组和指针之间是紧密联系的，指针和数组的运算往往可以互换使用。
- 函数指针可以做函数参数，函数也可以返回指针。